FH Salzburg
MultiMediaTechnology

Basics Game Development

Asset Handling, Rendering Pipeline - Lab

Technology
Health
Media

10 Minute Quiz

Assignment 1 Presentation

Today's Topics

- Asset Handling
  - Sounds / Music
  - Font / Text
- Singleton Pattern
- Input Handling
- Dictionaries

Initial Feedback Assignment 1

- Git Workflow:
  - Gitignore should include bin/, obj/ (have to delete files that are already committed first)
  - Commit more than once per Assignment
  - Commit messages should describe what was done
  - Be consistent in your style of message

- Coding Style:
  - Remove dead code (it is version controlled anyway; if you commit early and often)
  - Avoid "magic numbers"
  - Be consistent with your whitespace and indentation
  - Be consistent with your language
  - Make sure to remove temporary debug logs before publishing code

- Quick Tips:
  - SFML Vector classes have many operator overrides
  - Re-use your own Methods

Asset Handling – Sprites and Textures

Already known

```
Texture texture = new Texture(filePath);
Sprite sprite = new Sprite(texture);
```

Asset Handling – Sound and Soundbuffer

- A soundBuffer holds the data of sound, which is an array of audio samples

- Sound is the class used to play a "sound"

```
SoundBuffer soundbuffer = new SoundBuffer(fileName);
Sound Sound = new Sound(soundbuffer);
```

- Sound provides:
  - Control (play, pause, stop)
  - Ability to modify output parameters in real-time (pitch, volume, ...)
  - 3D spatial features (position, attenuation, ...)

https://www.sfml-dev.org/documentation/2.5.1/

Asset Handling – Music

- Music = sounds that are streamed rather than completely loaded in memory
- Especially useful for compressed music that usually take a lot memory when they are uncompressed
- Almost the same features as the Sound / SoundBuffer pair (play/pause/stop etc)

```
Music music = new Music(fileName);
```

https://www.sfml-dev.org/documentation/2.5.1/classsf_1_1Music.php#details

Asset Handling – Font & Text

- Font holds the data of a loaded font
- Text is the class to display fonts
- Output text has several options such as character size, style, color, position, rotation, etc.

```
Font font = new Font("C:/Windows/Fonts/arial.ttf");
Text text = new Text("Grundlagen Game Development!", font);
```

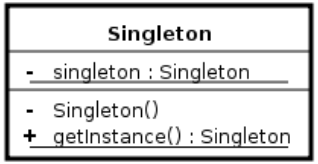
https://www.sfml-dev.org/documentation/2.5.1/

Singleton

```
/// <summary>
/// Simplified version of a Singleton
/// not thread safe - but we don't care
/// </summary>
3 references
public class InputManager
{
    3 references
    private static InputManager instance;
    0 references
    public static InputManager Instance
    {
        get
        {
            if (instance == null)
                instance = new InputManager();

            return instance;
        }
    }
    1 reference
    private InputManager(){}
}
```

- Software design pattern / anti-pattern
- Singleton is used to guarantee only a single Instantiation of a class exists
- Often used for "manager"-classes / to coordinate access and action across the system
- Dynamic Instance, no memory allocated at program startup (like in static classes)
- Call: `InputManager.Instance.MemberMethodOrField`



Input Handling – Window Events

```
    window.KeyPressed += OnKeyPressed;
    window.KeyReleased += OnKeyReleased;
}

1 reference
void OnKeyPressed(object sender, SFML.Window.KeyEventArgs e)
{
    if (e.Code == Keyboard.Key.W)
    {
        // Do stuff on key pressed
    }
}

1 reference
void OnKeyReleased(object sender, SFML.Window.KeyEventArgs e)
{
    if (e.Code == Keyboard.Key.W)
    {
        // Do stuff on key released
    }
}
```

Hash Maps

- Dictionaries in C#: https://docs.microsoft.com/en-us/dotnet/api/system.collections.generic.dictionary-2?view=net-6.0
- A container with Keys and Values

```
Dictionary<string,int> demoDict = new Dictionary<string, int>();
demoDict["Test"] = 1;

if(demoDict.ContainsKey("Test"))
    Console.WriteLine($"Test is in the Dictionary {demoDict["Test"]}");

foreach(KeyValuePair<string, int> item in demoDict)
    Console.WriteLine($"Key: {item.Key} Value: {item.Value}");
```

Input Handling – KeyDown, KeyUp, KeyPressed

- The keyPressed & keyReleased events are triggered when a keyboard key is pressed/released
- If a key is held, multiple KeyPressed events will be generated, at the default operating system delay
- Call **`window.setKeyRepeatEnabled(false)`** to disable this behavior
- On the flip side, it is obvious that KeyReleased events can never be repeated

https://www.sfml-dev.org/tutorials/2.5/window-events.php

Assignment 2

FH Salzburg
MultiMediaTechnology

Technology
Health
Media

© 2025 Fachhochschule Salzburg GmbH
www.fh-salzburg.ac.at